

Project: Machine Learning Analysis

1 Introduction

No generative AI tools may be used.

In this step of the project, you will apply regression or classification to predict an aspect of your project, as outlined in your revised projected proposal. This builds upon the work that you performed in the previous milestone in that data will have already “munged” and is ready to be used for training. As you work with the specific aspect of your project/problem, keep in mind the questions that you posed in the proposal. The research objectives are meant to be concise - they form the high-level goal. The questions are what you are trying to answer by applying predictive techniques.

You are welcome to apply any predictive technique supported by the packages in Section 2.4. This means you can use not just the techniques from class (e.g., linear regression, logistic regression, nearest neighbors, clustering, etc) but also techniques not covered (e.g., Random Forest, XGBoost, etc).

Thinking ahead: later in the semester you will be writing a paper that presents your data science project. In that paper, you will present your findings on the feasibility of your predictive technique to answer your questions (to satisfy your research objectives). This will require not just describing what technique you applied but also providing insight into how the model functions (i.e., how features play a role in the decision that is made).

1.1 Project Structure

In this part of the project, you will extend the workflow that you developed in the DM&V milestone. It must be structured as described in this section.

Source Code and Documentation: the five or six files from the DM&V milestone will remain in your project: `wf_datagen.py`, `wf_dataprocessing.py`, `wf_visualization.py`, `wf_core.py`, `project_exploration.md`, `requirements.txt`. You may update them if needed. `project_proposal_initial.md` and `project_proposal_revised.md` must also be included. The following new files will be added to your project:

- `wf_ml_evaluation.py`: This will contain code that splits your data into test and training sets (which be stored in the "models" folder), trains a model (by calling functions in `wf_training.py`), and then evaluates it by performing prediction on the test set (by calling functions in `wf_prediction.py`). This file provides a secondary entry point to running your workflow. It should be assumed that `wf_core.py` has previously executed to populate the “data_processed” folder.
- `wf_ml_training.py`: This will contain code that constructs one (or more) models from your data. Executing it will save file(s) which stores the model(s).
- `wf_ml_prediction.py`: This will contain code that uses the model stored in the folder "model" to perform a prediction.
- `project_ml_experimentation.md`: This will contain documentation from evaluating your model.
- `project_ml_evaluation.md`: This will contain documentation for experimenting with the model the workflow produces.

If you wish to further subdivide your .py work into additional files, use the above names as prefixes. For example: you would still have a `wf_ml_evaluation.py` file that processes your data, but it might call methods from `wf_ml_evaluation_kfold.py`. The data and visuals folders from the previous assignment will remain unchanged. Two new folders will be added: “models” and “evaluation”:

Dropbox > Git > ser494 > project_structure >		
Name	Date modified	Type
data_original	10/6/2022 11:41 PM	File folder
data_processed	10/6/2022 11:40 PM	File folder
evaluation	11/5/2022 6:27 PM	File folder
models	11/5/2022 6:27 PM	File folder
visuals	10/6/2022 11:39 PM	File folder
project_exploration.md	10/7/2022 10:27 AM	MD File
project_ml_evaluation.md	11/7/2022 9:58 PM	MD File
project_ml_experimentation.md	11/7/2022 9:58 PM	MD File
project_proposal_initial.md	10/6/2022 11:48 PM	MD File
project_proposal_revised.md	11/5/2022 6:27 PM	MD File
requirements.txt	10/7/2022 10:26 AM	Text Document
wf_core.py	10/6/2022 11:40 PM	Python File
wf_dataprocessing.py	10/6/2022 11:39 PM	Python File
wf_ml_evaluation.py	11/5/2022 6:28 PM	Python File
wf_ml_prediction.py	11/7/2022 10:00 PM	Python File
wf_ml_training.py	11/5/2022 6:28 PM	Python File
wf_visualization.py	10/6/2022 11:41 PM	Python File

Figure 1: Basic project file structure. Your project will contain all of these files and folders, and potentially a few more. `project_proposal_initial.md` will remain unchanged from MS1.

Data: from the DM&V milestone, the processed data will be stored in the sub-folder called “data_processed”. The file `wf_evaluation.py` will load this existing data and save training/test sets back into the file.

Model(s): the model(s) you produce must be stored in a sub-folder called “models”.

Evaluation: the evaluation you produce must be stored in a sub-folder called “evaluation”.

Visuals: this will remain from the DS&V milestone.

An example of the basic structure for your project is shown in Figure 1. The overall design for the completed workflow is shown in Figure 2.

2 Requirements

For this assignment, you will apply a machine-learning approach. All work must be implemented in Python 3.8 or newer. The model(s) files that you create should be placed in a sub-folder called “models”. The statistical evaluation will be placed in a sub-folder called “evaluation”. Using the data already in `data_processed`, the following three high-level steps must be followed: machine learning, statistical evaluation, and experimentation.

The program (workflow) that you construct must be designed in a way that others can execute it to generate the results. Make sure that paths are not hard coded and that you are running the most recent version of any packages used.

2.1 Machine Learning

1. Construct the training and test datasets.

- **What:** Create reasonable training and test datasets. At a minimum, you should ensure that each set is representative of the entire dataset and you use 80% of your data for training, and 20% for testing. Using a more advanced technique like k-fold validation may be useful but is not required. (Anyone using time sequence data should be careful here!)

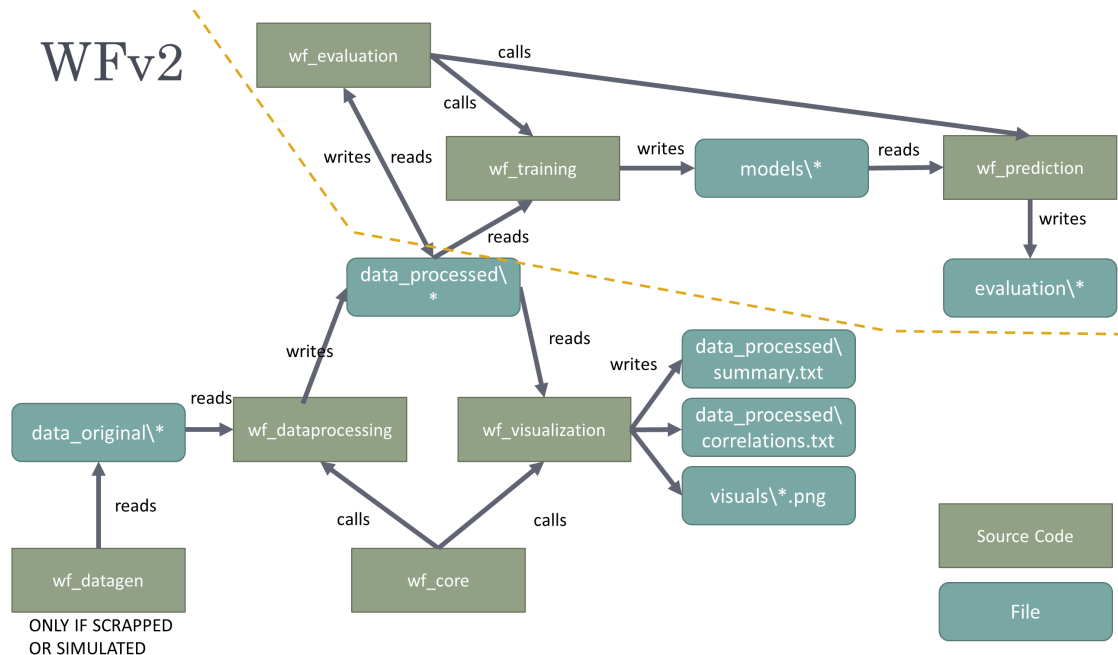


Figure 2: High-level workflow design. The highlighted region contains new system elements.

- As needed: standardize (e.g., Z-score) the data.
 - The test dataset must have at least 30 samples. (Hopefully you have much much more than that!) If you have less, please look at the techniques for increasing the size of a small dataset.
 - Where: code will go in `wf_ml_evaluation.py`, the initial input data will be read from “data_processed” and the output should be saved to “data_processed”.
2. Train a model on the training dataset.
 - What: Create a model for your data. As mentioned previously, you are welcome to apply any predictive technique supported by the packages in Section 2.4. It is not expected that you implement an algorithm from scratch here.
 - Where: code will go in `wf_ml_training.py`, and the model should be saved to the folder “models”.
 - **Note: Do not accidentally commit solutions to the machine learning homework (5 and later) in your repo.**
 3. Predict an output based on an input.
 - What: Using the model saved in the “models” folder, perform a prediction of a given input. This should be a relatively concise file. It will be used by the statistical evaluation code below.
 - Where: code will go in `wf_ml_prediction.py`. (Typically, it would load the model from “models”.)

2.2 Statistical Evaluation

1. Compute an evaluation for a model.
 - What: Pick two appropriate metrics for evaluating your type of prediction (e.g., accuracy for classification, MSE for regression), and compute them for a model (e.g., the one that was constructed). See the sample output on the second slide after the section on Evaluation Environments in the Mathematical Models module.
 - Where: code will go in `wf_ml_evaluation.py`, the output should be saved to “evaluation\summary.txt” (typically a row).

2. Construct alternative models for tuning.

- What: Construct multiple models (at least three) based on your training data and evaluate each. “Alternative models” is intended to be generic phrasing: we aren’t explicitly requiring a baseline model. (Of course: a baseline is an alternative model!) Tuning a model produces alternative models. See suggestions below.
- Where: code will go in `wf_ml_evaluation.py`, the output should be saved to “`evaluation\summary.txt`” (will call previous code to form a matrix).
- This step can potentially take a significant amount of time. We suggest you create different versions of the model you have already built, rather than construct something (e.g., a baseline) from scratch:
 - For a technique like linear regression, an alternative model can be created using something like ridge vs LASSO regression.
 - For a technique like KNN, it can be obvious what to do: create different models with different values of k . In this case, we just tune k , which can be done relatively quickly.
 - You also think about using a subset of features (should work for any algorithm), just don’t do something that will obviously lead to a worse score.
 - If you want (it may strengthen your project), you can try evaluating your model on different subsets of your data.

In addition, you will need to fill out the file `project_ml_evaluation.md` (see Canvas for a template file, remove “_template” from the name) with information on your evaluation:

1. Title: the name of your project.
2. Author: indicate your name.
3. Date: indicate the date of submission.
4. Evaluation Metrics: list the metrics you choose and explain why each of the metrics is appropriate for your problem.
5. Alternative Models: for each alternative model:
 - Describe how the model was constructed relative to your initial model.
 - Describe how the model fared during evaluation.
 - (If you created more than three alternatives, you are not required to describe them all. You should, however, be sure to include the best alternative you created.)
6. Best Model: list which of the alternative models was best performing.

2.3 Experimentation

As you work through the following, remember that we want to be able to explain (defend?) how the model works. (Don’t forget about meeting your outcomes from the last milestone!) Using your best model, perform the following:

1. Apply your model to inputs and evaluate the result.
 - What: select two (2) records from your data (maybe the DM&V ones). Explain why your model’s output is reasonable for those two sample inputs. **You must argue that the prediction provided by the model is correct in terms of the domain. Do not simply say that it matches the trend in the data (how could it not!).**
 - Where: `project_ml_experimentation.md`
2. Select interesting features.

- What: Select two (2) features (call them A and B) in the data that should have a significant impact on the prediction. (Imagine that these are parameters of a function that you want to test.). Describe and justify why (from domain knowledge) you think the model should be impacted by variation in those two features.
- Where: `project_ml_experimentation.md`

3. Experiment with the impact of features on the output.

- What: Your goal is to describe the (hopefully causal) mechanisms that are implied by the way your model interacts with an input.
Select **two** features (call them A and B) in the data that should have a significant impact on the prediction. (Imagine that these are parameters of a function that you want to test.)
 - Start by engineering¹ a set of sample inputs that vary only across feature A. Run these and review the predictions. What trend emerges?
 - Repeat with feature B. What trend emerges?
 - Repeat with varying A and B together: correlated and inversely. What trend emerges? In particular, is there any change not present when A and B changes alone?
- Where: `project_ml_experimentation.md`. If needed, you can add additional code in `wf_ml_evaluation.py`.

You will need to fill out the file `project_exploration.md` (see Canvas for a template file, remove “_template” from the name) with information on your experimental process:

1. Title: the name of your project.
2. Author: indicate your name.
3. Date: indicate the date of submission.
4. Explainable Records: for each of the two records:
 - (a) Give the record (or the path if an image or bitstream).
 - (b) The explanation.
5. Interesting Features: for each of the two features:
 - (a) Give the feature.
 - (b) The justification.
6. Experiments: for each of the four possible variations, list the trend in prediction that you observe.

2.4 Allowed Packages

For this milestone, the following packages are allowed:

- Anything available by default on a Python 3.8+ install.
- Matplotlib, Numpy, Pandas, Requests, BeautifulSoup4, NLTK, OpenCV, Seaborn
- (NEW) scikit-learn (sklearn)
- (NEW) scipy (scipy)
- (NEW) TensorFlow (tensorflow)

We may allow use of other packages on a case by case basis. Please check with the instructor or TA at least 72 hours prior to the deadline.

¹You can also pick out other inputs that are close to your initial one but differ in the appropriate dimension.

2.4.1 requirements.txt

Update your requirements.txt as done in the DM&V milestone.

3 Deliverables

For this assignment, you will receive one grade, which evaluates:

- Project: Machine Learning Analysis Submission (Canvas, GitHub): see Section 2 for details. [40 points]
 - The results of your analysis must be provided in the two markdown files.
 - Your work should be: 1) uploaded to your GitHub repository, and 2) attached to the Canvas assignment (compress and upload your repository). Both are due by the deadline on Canvas and must be identical.

4 Next Steps

See Canvas for the next assignment:

1. MS6: Paper